Task 4 – Event-Handling Forensics: Drawing Pad (7 Marks, K4)

Run each program yourself and attach your own screenshots, each linked to the requirement or test it shows. Items marked `<fill>` need your own observations or measurements.

Task 4.1 – Locate the defects

Why the click is never handled: the method is spelled `mouseclicked` (lowercase c), but `MouseAdapter` declares `mouseClicked`. Java is case-sensitive, so the method is **not an override**. It is simply a new, never-called method. The adapter's own empty `mouseClicked` runs instead, so nothing happens.

Why the label never changes: two independent reasons. (1) The handler never runs (above). (2) Even if it ran, it only does `count++` and never calls `status.setText(...)`, so the label text stays "Clicks: 0".

(Related problem: drawing with `getGraphics()` inside the handler draws on a temporary graphics context, which is erased at the next repaint; see 4.2.)

Task 4.2 – A failing interaction

Even after fixing the method name: **click a few dots, minimise the window, then restore it** (or drag another window over it). The dots disappear.

Why: AWT/Swing windows repaint themselves by calling `paint(Graphics)`. Whatever was drawn earlier with `getGraphics()` was only painted onto the screen once and is not stored anywhere. When the window is restored, the system calls `paint()`, which draws only the background, because the program never remembered the points. The fix is to store the points and draw them all inside `paint()`.

Task 4.3 – Repaired program (DrawPadFixed.java)

```
import java.awt.*;
import java.awt.event.*;
import java.util.ArrayList;
import java.util.List;

public class DrawPadFixed extends Frame {
    private final List<Point> points = new ArrayList<>();
    private final Label status = new Label("Clicks: 0");

    public DrawPadFixed() {
        super("Draw Pad");
        add(status, BorderLayout.SOUTH);
        addMouseListener(new MouseAdapter() {
            @Override public void mouseClicked(MouseEvent e) {
                points.add(e.getPoint());
                status.setText("Clicks: " + points.size());
```

```
                repaint();
            }
        });
        addWindowListener(new WindowAdapter() {
            @Override public void windowClosing(WindowEvent e) {
dispose(); }
        });
        setSize(400, 300);
    }

    @Override public void paint(Graphics g) {
        super.paint(g);
        for (Point p : points) g.fillOval(p.x - 3, p.y - 3, 6, 6);
    }

    public static void main(String[] args) { new
DrawPadFixed().setVisible(true); }
}
```

Task 4.4 – Prevention with `@Override`

The given broken code has **no** `@Override` on the misspelled method, so it compiles without any message and the bug only shows at run time. With `@Override` written on the method, the compiler checks that it really overrides something in the superclass and reports the mistake immediately. Actual compiler output when `@Override` is present and the name is misspelled `mouseclicked`:

```
DrawPad.java:8: error: method does not override or implement a method
from a supertype
      @Override public void mouseclicked(MouseEvent e) {
      ^
1 error
```

Using `@Override` on every listener method turns this class of mistake into a compile-time error.

Complete the Code

TODO 1 (inside the `MouseAdapter`)

```
@Override public void mouseClicked(MouseEvent e) {
    points.add(e.getPoint());
    status.setText("Clicks: " + points.size());
    repaint();
}
```

TODO 2 (inside `paint`)

```
super.paint(g);
for (Point p : points) g.fillOval(p.x - 3, p.y - 3, 6, 6);   // 6 x 6
circle centred on the point
```

(Only the code at the two TODO markers changes; the rest of the given program stays as it is. The program in 4.3 is the same code with these two blocks filled in.)

Experiment to record: (1) misspell the method `mouseclicked` and remove `@Override`: it compiles with no message, but clicks do nothing. (2) Keep `@Override` with the misspelled name: the compiler error above appears (this is the compiler message to record). (3) Restore the correct name `mouseClicked`. Take screenshots before and after minimising the window; with the repaired program the dots are still there after restoring. `<fill>`

Alternative concept and complexity

Draw once onto an off-screen `BufferedImage`, copy the image in `paint()`:

```
private final BufferedImage canvas = new BufferedImage(400, 300,
BufferedImage.TYPE_INT_ARGB);

// in mouseClicked:
Graphics2D g2 = canvas.createGraphics();
g2.fillOval(e.getX() - 3, e.getY() - 3, 6, 6);
g2.dispose();
repaint();

@Override public void paint(Graphics g) { super.paint(g);
g.drawImage(canvas, 0, 0, null); }
```

- **Time per repaint for n clicks:** stored points redraw n ovals every time: O(n), so repaints get slower the more dots there are. The image version does one `drawImage` copy of the fixed-size image: O(width × height) = O(1) with respect to n. Drawing a new dot costs O(1) in both designs.
- **Space:** stored points need O(n) memory (one `Point` per click). The image needs a constant O(width × height) pixels (about 4 bytes per pixel with ARGB, so 400 × 300 × 4 ≈ 480 KB), regardless of the number of clicks.
- **Trade-off:** the point list is better for few clicks and when you need to undo, move or export individual points. The image is better for very many strokes, but you lose the individual objects and must handle window resizing (a new, larger image). Own measurement: time `paint()` with n = 1,000 / 100,000 points `<fill>`.